

***SIMATS ENGINEERING
CO2-ASSESSMENT TOOL 1***

***COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1103***

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: Any 4

Total Marks:40

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

***I GIRIJA B/192311044 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.***

Signature of the student: GIRIJA B

ANALYTICAL PROBLEM SOLVING

1. Use Case Diagram Problem

A Hospital Management System allows patients to book appointments, doctors to update medical records, and administrators to manage hospital resources. The requirements are not clearly documented.

Tasks:

- Identify all actors involved in the system.
- List the primary use cases.
- Define relationships (include, extend, generalization).
- Construct a complete Use Case Diagram representing system functionality.

(a) Problem Understanding: The Hospital Management System is designed to automate hospital operations. It enables:

- Patients to book and manage appointments.
- Doctors to access and update patient medical records.
- Administrators to manage hospital resources such as doctors, staff, rooms, and equipment.

The requirements indicate three main user groups interacting with the system.

(b) Identify All Actors

Actor	Responsibility
Patient	Registers, logs in, books appointments, views appointments
Doctor	Views appointments, updates medical records, prescribes treatment
Administrator	Manages doctors, patients, hospital resources, appointments, reports

(c) Primary Use Cases

Patient

- Register, Login, Book Appointment, View Appointment, Cancel Appointment

Doctor

- Login, View Appointment Schedule, Update Medical Record, Prescribe Medicine

Administrator

- Login, Manage Doctors, Manage Patients, Manage Hospital Resources, Generate Reports, Manage Appointments

(d) Relationships (Include, Extend, Generalization)

Include (Mandatory Relationship)

- **Book Appointment** <<include>> **Login**
- **Update Medical Record** <<include>> **Login**
- **Manage Hospital Resources** <<include>> **Login**
- **Generate Reports** <<include>> **Login**

Extend (Optional Relationship)

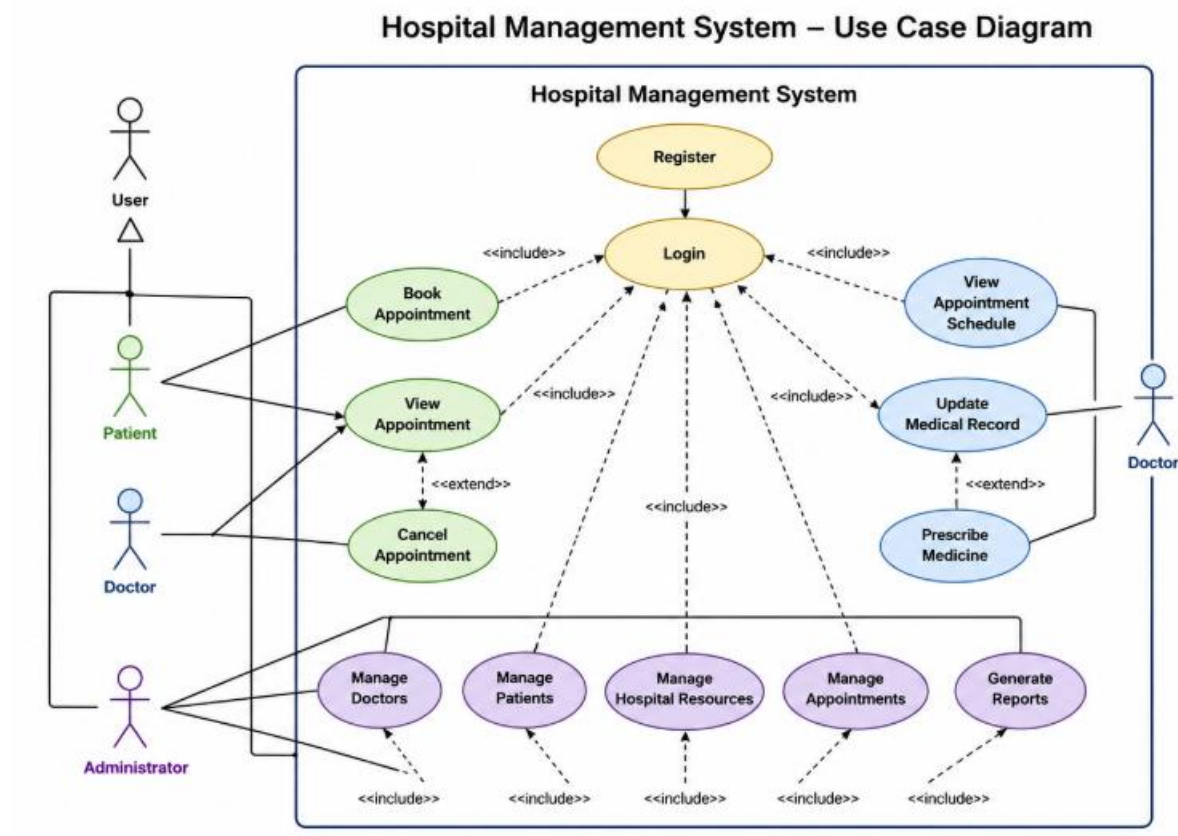
- **Cancel Appointment** <<extend>> **View Appointment**
- **Prescribe Medicine** <<extend>> **Update Medical Record**

Generalization: User (General Actor)

- Patient
- Doctor
- Administrator

All inherit common functionality such as Login.

(e) Complete Use Case Diagram



2. Class Diagram Problem

A Library Management System contains entities such as Book, Member, Librarian, and Issue, Record. The relationships among entities are not properly defined.

Tasks:

- a) Identify classes with attributes and methods.
- b) Define relationships (association, aggregation, inheritance).
- c) Specify multiplicity between classes.
- d) Design a detailed Class Diagram for the system.

(a) Problem Understanding

The Library Management System manages books, members, librarians, and issue records. The system keeps track of books issued and returned by members. A librarian is responsible for issuing and receiving books and maintaining library records.

(b) Identify Classes with Attributes and Methods

1. Book: Attributes: bookID : int , title : String , author : String , publisher : String, status : String

Methods: addBook(), updateBook(), searchBook(), removeBook()

2. Member: Attributes: memberID : int, name : String , address : String, phone : String

Methods: borrowBook(), returnBook(), viewBooks()

3. Librarian: Attributes: librarianID : int, name : String, email : String

Methods: issueBook(), receiveBook(), manageBooks()

4. IssueRecord: Attributes: issueID : int, issueDate : Date, returnDate : Date, fine : double

Methods: calculateFine(), updateRecord()

(c) Relationships

Association: Member ↔ IssueRecord, Book ↔ IssueRecord, Librarian ↔ IssueRecord

Aggregation: Librarian ◇— Book

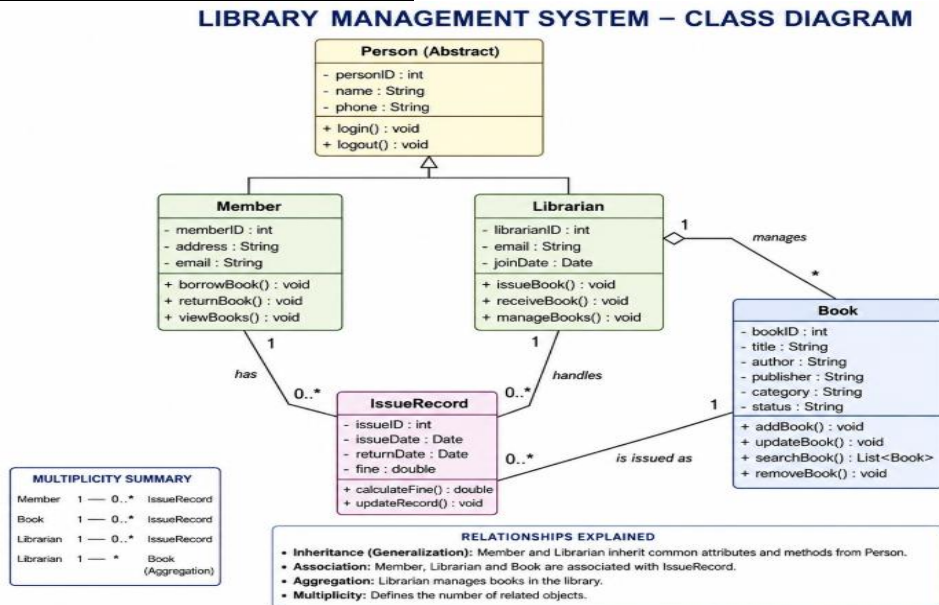
A librarian manages many books, but books can exist independently.

(d) Multiplicity

Relationship	Multiplicity
Member — IssueRecord	1 ---- 0..*
Book — IssueRecord	1 ---- 0..*

Relationship	Multiplicity
Librarian — IssueRecord	1 ---- 0..*
Librarian — Book	1 ---- *
Person — Member	Generalization
Person — Librarian	Generalization

(e)UML Class Diagram Image



3. Sequence Diagram Problem

In an Online Food Ordering System, a customer selects food items, places an order, makes payment, and receives order confirmation. The interaction flow is unclear.

Tasks:

- Identify objects involved in the process.
- Define message flow between objects.
- Represent activation and lifelines.
- Construct a Sequence Diagram showing complete interaction flow.

(a) Problem Understanding

The Online Food Ordering System allows a customer to browse the menu, select food items, place an order, make payment, and receive an order confirmation. The interaction between the customer, system, payment gateway, restaurant, and notification service must be represented in the correct sequence.

(b) Identify Objects Involved

Object	Responsibility
Customer	Selects food, places order, makes payment
User Interface (UI)	Displays menu and communicates with services
Order Service	Creates and manages orders
Payment Gateway	Processes online payments
Restaurant Service	Receives and accepts the order
Notification Service	Sends order confirmation to the customer

(c) Define Message Flow

1. Customer → User Interface : Browse Menu
2. User Interface → Customer : Display Menu
3. Customer → User Interface : Add Items to Cart
4. Customer → User Interface : Place Order
5. User Interface → Order Service : Create Order
6. Order Service → User Interface : Order Created
7. Customer → User Interface : Make Payment
8. User Interface → Payment Gateway : Process Payment
9. Payment Gateway → User Interface : Payment Status
10. Order Service → Restaurant Service : Send Order
11. Restaurant Service → Order Service : Order Accepted
12. Order Service → Notification Service : Send Confirmation
13. Notification Service → User Interface : Confirmation Sent
14. User Interface → Customer : Display Order Confirmation

(d) Activation and Lifelines

Lifelines: Customer, User Interface, Order Service, Payment Gateway, Restaurant Service, Notification Service

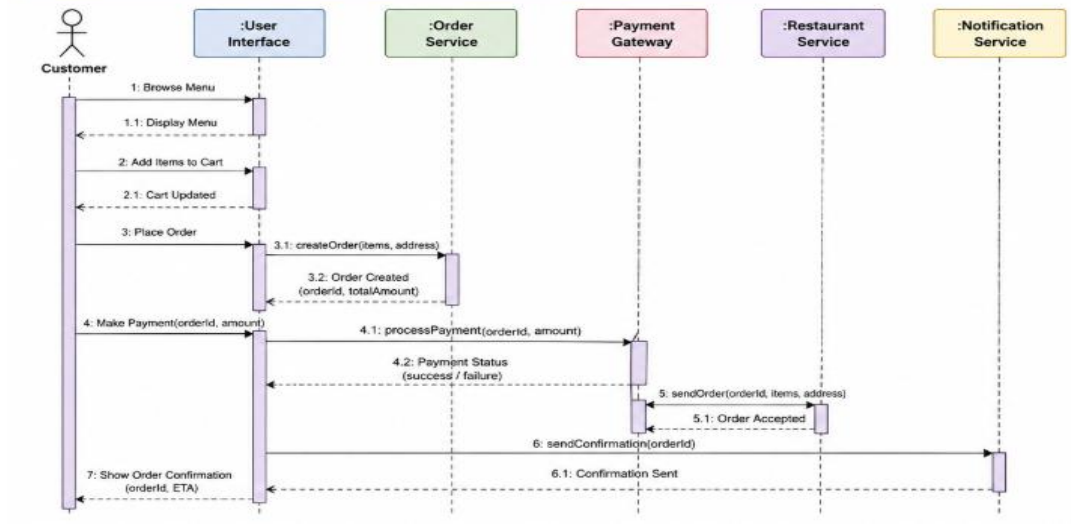
Activations

- User Interface is activated when the customer interacts with the application.
- Order Service is activated during order creation.
- Payment Gateway is activated while processing payment.
- Restaurant Service is activated when receiving the order.
- Notification Service is activated while sending confirmation.

(e) Complete Sequence Diagram

ONLINE FOOD ORDERING SYSTEM – SEQUENCE DIAGRAM

Scenario: Customer selects food items, places order, makes payment and receives order confirmation.



4. Activity Diagram Problem

A Student Admission System includes application submission, document verification, fee payment, and enrollment confirmation. The workflow lacks proper decision-making steps.

Tasks:

- Identify all activities in the workflow.
- Add decision nodes for approval/rejection.
- Include parallel processing where applicable.
- Design an optimized Activity Diagram.

(a) Problem Understanding

The Student Admission System automates the admission process from application submission to enrollment confirmation. The workflow includes submitting the application, verifying documents, paying admission fees, and confirming enrollment. Decision nodes are required to handle application approval/rejection and payment success/failure. Parallel processing improves efficiency by verifying documents and checking eligibility simultaneously.

(b) Identify All Activities

Activity	Description
Register / Create Account	Student creates an account
Fill Application Form	Student enters personal and academic details
Upload Documents	Student uploads required certificates
Submit Application	Application is submitted to the system
Validate Application	System checks application completeness

Activity	Description
Verify Documents	Admission officer verifies uploaded documents
Check Eligibility	Admission officer verifies eligibility criteria
Approve/Reject Application	Admission decision is taken
Pay Admission Fee	Student pays the admission fee
Process Payment	Finance department verifies payment
Generate Receipt	Payment receipt is generated
Confirm Enrollment	Student admission is confirmed
Generate Student ID	Student ID is created
Send Confirmation	Admission confirmation is sent to the student

(c) Decision Nodes

Decision 1: Application Complete?

- **Yes** → Forward application for verification.
- **No** → Notify applicant to correct or complete the application.

Decision 2: Documents & Eligibility Approved?

- **Yes** → Proceed to fee payment.
- **No** → Reject application and notify applicant.

Decision 3: Payment Successful?

- **Yes** → Generate receipt and confirm enrollment.
- **No** → Notify payment failure and request another payment attempt.

(d) Parallel Processing (Fork and Join)

Fork Node

After application validation, two activities occur simultaneously:

- Verify Documents
- Check Eligibility

Join Node

Both parallel activities must finish before the system decides whether to approve or reject the application.

(e) Optimized Activity Diagram

